

Regras de Negócio - Projeto de Banco de Dados

JOÃO RAFAEL DE SOUZA DANTAS

Professor: Hertz Wilton De castro Lins

20/05/2025

Documentação de Regras de Negócio e Arquitetura de Dados

Projeto: Sistema de Cadastro e Cálculo de Salário

Database: calculo_salario2

1. Visão Geral

Este documento descreve as regras de negócio que governam o sistema de cadastro de pessoas e o cálculo de seus respectivos salários. O projeto adota uma arquitetura onde a lógica de negócio principal é centralizada no banco de dados através de *Stored Procedures* e *Views*, garantindo consistência, segurança e desacoplamento da camada de aplicação.

2. Modelo de Dados

O banco de dados é composto por três tabelas principais e uma view de consulta:

- **cargo:** Tabela de domínio que armazena os diferentes cargos existentes na organização e seu respectivo salário base.
- **pessoa:** Tabela principal que armazena os dados cadastrais e profissionais dos funcionários.
- **pessoa_salario:** Tabela transacional que armazena o resultado do cálculo de salário para cada pessoa em um determinado mês/ano.
- **vw_pessoa_salario_ativo:** View que serve como a única interface de consulta para a aplicação, expondo os dados salariais apenas dos funcionários ativos.
- **Contador_matricula:** Entidade responsável por manter o controle da criação das matrículas, funcionando de forma sequencial e por ano.

3. Regras de Negócio e Implementação Técnica

A seguir, detalhamos cada regra de negócio e como ela foi implementada na estrutura do banco de dados.

3.1. Gestão de Pessoas (Tabela pessoa)

RN01: Obrigatoriedade de Dados Cadastrais

- **Descrição:** Todos os campos para o cadastro de uma nova pessoa são de preenchimento obrigatório.

- **Implementação:** Todos os campos na tabela pessoa (exceto ativo, que tem um valor padrão) são definidos com a constraint NOT NULL. O banco de dados rejeitará qualquer tentativa de inserção ou atualização que deixe um desses campos nulo.

RN02: Unicidade de E-mail, Usuário e CPF

- **Descrição:** Cada pessoa cadastrada no sistema deve possuir um endereço de e-mail, um usuário e um cpf único. Não é permitido cadastrar duas pessoas com qualquer um desses atributos repetidos.
- **Implementação:** O campo email, usuario e cpf, na tabela pessoa possui uma constraint UNIQUE KEY. O banco de dados automaticamente impede a inserção de um registro com quaisquer desses que já exista na tabela.

RN03: Exclusão Lógica

- **Descrição:** Os registros de pessoas não são fisicamente deletados do banco de dados. Em vez disso, eles são marcados como inativos. Isso preserva o histórico de todos que já passaram pela organização.
- **Implementação:** A tabela pessoa contém a coluna ativo TINYINT(1).
 - ativo = 1: O registro está ativo e é considerado válido para todas as operações de negócio (ex: cálculo de salário).
 - ativo = 0: O registro está inativo ("excluído" logicamente) e não deve ser considerado em novas operações.
-

RN04: Inserção de Novas Pessoas via Procedure

- **Descrição:** A criação de um novo registro de pessoa deve ser feita exclusivamente através de um procedimento padrão (É possível inserir via banco, mas requer conhecimento avançado de banco de dados e não é o recomendado).
- **Implementação:** A Stored Procedure inserir_pessoa(...) foi criada para encapsular a lógica de inserção. Ela recebe todos os dados da pessoa como parâmetros e insere o registro na tabela pessoa, definindo o campo ativo como 1 (ativo) por padrão.

3.2. Cálculo de Salário (Tabela pessoa_salario e Lógica Associada)

RN05: Fórmula do Salário Líquido

- **Descrição:** O salário líquido de um funcionário é calculado pela fórmula: Salário Líquido = Salário Base + Bônus - Descontos.
- **Implementação:** Esta regra é implementada diretamente pela view, não havendo inserção no banco físico, e recalculado dinamicamente a cada nova consulta.

- Isso garante que o valor do `salario_liquido` seja **sempre** consistente com os demais valores, sendo calculado e mantido pelo próprio SGBD, eliminando a possibilidade de erro por parte da aplicação, e também garantindo a normalização, eliminando dependências transitivas nas entidades.

RN06: Processo de Cálculo Mensal de Salários

- **Implementação:** A Stored Procedure `calcular_salarios(IN p_bonus DECIMAL(10,2))` centraliza toda essa lógica:
 1. Ela recebe um percentual de bônus como parâmetro (`p_bonus`) e um desconto(`p_descontos`).
 2. Primeiro, ela limpa (`TRUNCATE TABLE`) todos os registros da tabela `pessoa_salario` para garantir que apenas o cálculo atual permaneça.
 3. Em seguida, ela insere um novo registro em `pessoa_salario` para **cada pessoa com ativo = 1**.
 4. O `salario_base` é obtido da tabela `cargo`, com base no cargo da pessoa.
 5. O valor do bonus é calculado como `salario_base * (p_bonus / 100)`.
 6. O campo `descontos` é inserido também via aplicação.
-

3.3. Arquitetura de Acesso a Dados

RN07: Abstração de Acesso à Lógica de Negócio

- **Descrição:** A aplicação externa (seja um sistema web, desktop ou API) não possui permissão para acessar ou manipular as tabelas diretamente. Toda a interação com a lógica de negócio é feita através de uma camada de abstração no banco de dados.
- **Implementação:**
 - **Para Consultas:** A aplicação deve consultar **exclusivamente a view `vw_pessoa_salario_ativo`**. Esta view já contém a junção das tabelas `pessoa` e `pessoa_salario` e o filtro `WHERE p.ativo = 1`. Isso cumpre a regra de que "apenas registros ativos aparecem nas consultas" e impede que a aplicação tenha que conhecer a estrutura complexa das tabelas.
 - **Para Operações (Escrita):** A aplicação não executa comandos `INSERT`, `UPDATE` ou `DELETE`. Em vez disso, ela chama as Stored Procedures:
 - `CALL inserir_pessoa(...)` para cadastrar um novo funcionário.
 - `CALL calcular_salarios(...)` para disparar o processo de cálculo da folha de pagamento.

4. Resumo da Arquitetura de Interação

- 1. Aplicação quer cadastrar um funcionário:**
 - Chama CALL `inserir_pessoa(...)` com os dados do novo funcionário.
- 2. Sistema precisa calcular a folha de pagamento (ex: com 10% de bônus e descontos totais de 499.90):**
 - Chama CALL `calcular_salarios(10.0, 499.90)`.
- 3. Aplicação quer exibir o relatório de salários do mês:**
 - Executa `SELECT nome, email, salario, cargo FROM vw_pessoa_salario_ativo;`